

ENSEEIH7

RAPPORT PROJET SEC

MINISHELL

Aurélien VERBEKE

Année 2024 - 2025



Table des matières

I	Introduction	2
II	Processus	2
III	Signaux	3
IV	Fichiers et redirections	6
V	Tubes	8
VI	Remarques supplémentaires	8

I – Introduction

Le MINISHELL est un shell minimaliste qui permet de lancer des commandes, changer de répertoire courant, afficher le contenu d'un répertoire.

Les commandes peuvent être lancées en arrière-plan et chaînées, et l'utilisateur peut faire des redirections d'entrées / sorties.

II – Processus

Etape 2

On ajoute `execvp(commande[0], commande)`. Cela permet de remplacer le code courant par celui de la commande.

On utilise la version `vp` pour pouvoir transmettre tous les arguments en une seule fois et pour pouvoir utiliser le `PATH`.

Etape 3

Dans le père, on ajoute la directive `wait()` pour attendre la terminaison du fils lancé.

Etape 4

Dans le père, on remplace `wait()` par

```
if (ligneCommande->backgrounded == NULL) {
    int status;
    if (waitpid(pidFils, &status, 0) == -1) {
        exit(EXIT_FAILURE);
    }
}
```

On attendra alors que l'exécution des fils en avant-plan se termine.

Il est nécessaire d'effectuer un `waitpid()` sur le fils lancé pour ne pas que le traitement du père soit rétabli après la terminaison du fils en arrière-plan.

III – Signaux

Question

Voir étape 4.

Etape 5

On ajoute en début de `main()` :

```
struct sigaction actionFilsTermine;
actionFilsTermine.sa_handler = handlerFilsTermine;
actionFilsTermine.sa_flags = SA_RESTART;
sigemptyset(&actionFilsTermine.sa_mask);
if (sigaction(SIGCHLD, &actionFilsTermine, NULL) == -1) {
    exit(EXIT_FAILURE);
}
```

La fonction de traitement est

```
void handlerFilsTermine() {
    fprintf(stderr, "Handler : Le fils a change d'etat\n");
}
```

Etape 6

On peut remplacer la fonction de traitement par

```
void handlerFilsTermine() {
    int status;
    pid_t pid = waitpid(-1, &status, WNOHANG|WUNTRACED|WCONTINUED);

    fprintf(stderr, "Handler : Fils %d termine\n", pid);
}
```

Etape 7

On remplace le `waitpid()` du traitement du père par un simple `pause()`.

On constate que le terminal rend la main après le `sleep 10`. C'est normal puisque on attend n'importe quel signal, en particulier un pouvant venir d'un fils en arrière plan.

Etape 8

L'envoi du signal `SIGSTOP` fait entrer le processus dans un état de pause (T+).

L'envoi du signal `SIGCONT` relance le processus (état S+ puisque `sleep`).

Etape 9

On remplace le simple affichage de la fonction de traitement `handlerFilsTermine()` par :

```
if (WIFEXITED(status)) {
    fprintf(stderr, "Handler : Fils %d termine avec le code %d\n",
               pid, WEXITSTATUS(status));
} else if (WIFSTOPPED(status)) {
    fprintf(stderr, "Handler : Fils %d suspendu\n", pid);
} else if (WIFCONTINUED(status)) {
    fprintf(stderr, "Handler : Fils %d repris\n", pid);
} else if (WIFSIGNALED(status)) {
    fprintf(stderr, "Handler : Fils %d tue\n", pid);
}
```

Résultats

Pour le lancement de `sleep 60` et les envois consécutifs des signaux `SIGSTOP` et `SIGCONT` :

```
> Handler : Fils 48136 suspendu
Handler : Fils 48136 repris
Handler : Fils 48136 termine avec le code 0
```

```
ave9761@forge:~/git_perso/1a/sec/rapport$ ps -fjn
PPID    PID    PGID    SID TTY    TPGID STAT  UID    TIME COMMAND
41282   41289   41289   41289 pts/1    48123 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
41289   41292   41292   41289 pts/1    48123 S    57562   0:00 \_ /bin/bash -i
41292   48123   48123   41289 pts/1    48123 S+    57562   0:00 \_ ./minishell
48123   48136   48123   41289 pts/1    48123 S+    57562   0:00 \_ sleep 20
40174   40176   40176   40176 pts/0    48146 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
40176   40179   40179   40176 pts/0    48146 S    57562   0:00 \_ /bin/bash -i
40179   48146   48146   40176 pts/0    48146 R+    57562   0:00 \_ ps -fjn
ave9761@forge:~/git_perso/1a/sec/rapport$ kill -STOP 48136
ave9761@forge:~/git_perso/1a/sec/rapport$ ps -fjn
PPID    PID    PGID    SID TTY    TPGID STAT  UID    TIME COMMAND
41282   41289   41289   41289 pts/1    48123 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
41289   41292   41292   41289 pts/1    48123 S    57562   0:00 \_ /bin/bash -i
41292   48123   48123   41289 pts/1    48123 S+    57562   0:00 \_ ./minishell
48123   48136   48123   41289 pts/1    48123 T+    57562   0:00 \_ sleep 20
40174   40176   40176   40176 pts/0    48167 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
40176   40179   40179   40176 pts/0    48167 S    57562   0:00 \_ /bin/bash -i
40179   48167   48167   40176 pts/0    48167 R+    57562   0:00 \_ ps -fjn
ave9761@forge:~/git_perso/1a/sec/rapport$ kill -CONT 48136
ave9761@forge:~/git_perso/1a/sec/rapport$ ps -fjn
PPID    PID    PGID    SID TTY    TPGID STAT  UID    TIME COMMAND
41282   41289   41289   41289 pts/1    48123 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
41289   41292   41292   41289 pts/1    48123 S    57562   0:00 \_ /bin/bash -i
41292   48123   48123   41289 pts/1    48123 S+    57562   0:00 \_ ./minishell
48123   48136   48123   41289 pts/1    48123 S+    57562   0:00 \_ sleep 20
40174   40176   40176   40176 pts/0    48188 Ss    57562   0:00 bash -c source ~/.bashrc; $SHELL -i
40176   40179   40179   40176 pts/0    48188 S    57562   0:00 \_ /bin/bash -i
40179   48188   48188   40176 pts/0    48188 R+    57562   0:00 \_ ps -fjn
```

Etape 10

Que le processus soit en arrière-plan ou en avant-plan, on tue le minishell et la commande lancée.

Etape 11.1

On ajoute :

```
struct sigaction actionCtrlCZ;
actionCtrlCZ.sa_handler = handlerCtrlCZ;
actionCtrlCZ.sa_flags = SA_RESTART;
sigemptyset(&actionCtrlCZ.sa_mask);

if (sigaction(SIGTSTP, &actionCtrlCZ, NULL) == -1) {
    exit(EXIT_FAILURE);
}
if (sigaction(SIGINT, &actionCtrlCZ, NULL) == -1) {
    exit(EXIT_FAILURE);
}
```

La fonction de traitement est :

```
void handlerCtrlCZ() {
    fprintf(stderr, "Signal ignore par le pere.");
}
```

Le minishell bloque alors tout Ctrl-C et Ctrl-Z pour les fils et lui-même, ce qui est problématique.

Etape 11.2

De même qu'à l'étape précédente, on associe le traitement SIG_IGN aux signaux SIGTSTP et SIGINT avant le `fork()`.

Il faut alors, dans le fils, réassocier les traitements initiaux avec SIG_DFL aux signaux SIGTSTP et SIGINT pour qu'ils continue à les recevoir.

Pour plus de cohérence, on continue d'utiliser la fonction `sigaction()`.

Le résultat obtenu est celui qui était attendu : seulement les processus fils sont suspendus ou stoppés.

Etape 11.3

On a :

```
sigset_t masquePere;
sigset_t masqueFils;

sigemptyset(&masqueFils);

sigemptyset(&masquePere);
sigaddset(&masquePere, SIGINT);
sigaddset(&masquePere, SIGTSTP);
```

Et on ajoute `sigprocmask(SIG_SETMASK, &masquePere, NULL);` avant le `fork()` et `sigprocmask(SIG_SETMASK, &masqueFils, NULL);` dans le fils.

Le comportement est alors le même qu'à l'étape précédente, comme espéré.

Etape 12

On effectue `setpgrp()` dans le fils uniquement dans le cas où la commande est lancée en arrière-plan. Le fils est alors détaché du groupe de son père et ne reçoit plus les signaux correspondant à Ctrl-C et Ctrl-Z.

IV – Fichiers et redirections

Etape 13

Pour la redirection en entrée : il faut ouvrir le fichier donné (`fdSource = open(ligneCommande->in, O_RDONLY)`) et remplacer le descripteur de l'entrée standard par celui du fichier ouvert (`dup2(fdSource, 0)`).

Pour la redirection en sortie : il faut ouvrir le fichier donné et le créer s'il n'existe pas (`fdDest = open(ligneCommande->out, O_WRONLY|O_CREAT|O_TRUNC, 0644)`) et remplacer le descripteur de la sortie standard par celui du fichier ouvert (`dup2(fdDest, 1)`).

Etape 14

On crée d'abord une fonction qui change de répertoire :

```
void changerRepertoireCourant(const char* _nouveauRepertoire) {
    char erreur[512];
    char* nouveauRepertoire;

    // pas de repertoire fourni, on va dans le HOME
    if (_nouveauRepertoire == NULL) {
        nouveauRepertoire = getenv("HOME");
        sprintf(erreur, "Erreur changement repertoire vers home\n");
    }

    // on va dans le repertoire fourni
    else {
        nouveauRepertoire = malloc(256*sizeof(char));
        strcpy(nouveauRepertoire, _nouveauRepertoire);
        sprintf(erreur, "Erreur changement repertoire vers %s\n",
                nouveauRepertoire);
    }

    // execution du changement de repertoire
    if (chdir(nouveauRepertoire) == -1) {
        fprintf(stderr, erreur);
        exit(EXIT_FAILURE);
    }
}
```

On ajoute ensuite la prise en compte de la commande dans la boucle principale :

```
else if (strcmp(commande[0], "cd") == 0) {
    changerRepertoireCourant(commande[1]);
}
```

Enfin, on modifie l'affichage du prompt pour savoir où on est :

```
printf("(%)s) > ", getcwd(NULL, 0));
```

Etape 15

De même, on crée d'abord une fonction qui affiche un répertoire :

```
void afficherRepertoire(const char* _repertoire) {
    char erreur[512];
    char* repertoire;
    DIR* entreesRepertoire;
    struct dirent* entreeRepertoire;

    // pas de repertoire fourni, on affiche le repertoire courant
    if (_repertoire == NULL) {
        repertoire = getcwd(NULL, 0);
        sprintf(erreur, "Erreur ouverture repertoire courant\n");
    }

    // on affiche le repertoire fourni
    else {
        repertoire = malloc(256*sizeof(char));
        strcpy(repertoire, _repertoire);
        sprintf(erreur, "Erreur ouverture repertoire %s\n", repertoire);
    }

    // ouverture du repertoire
    if ((entreesRepertoire = opendir(repertoire)) == NULL) {
        fprintf(stderr, erreur);
        exit(EXIT_FAILURE);
    }

    // affichage du contenu
    while ((entreeRepertoire = readdir(entreesRepertoire)) != NULL) {
        printf("%s\n", entreeRepertoire->d_name);
    }

    // fermeture du repertoire
    if (closedir(entreesRepertoire) == -1) {
        fprintf(stderr, "Erreur fermeture repertoire\n");
        exit(EXIT_FAILURE);
    };
}
```

Puis on ajoute la prise en compte de la commande dans la boucle principale :

```
else if (strcmp(commande[0], "dir") == 0) {
    afficherRepertoire(commande[1]);
}
```

V – Tubes

Etapes 16 et 17

Avant la création du fils pour l'exécution de la commande, si une commande pipelinée suit :

— On sauvegarde le tube précédent

— On crée un nouveau tube

Puis, dans le fils :

— Une commande pipelinée suit : on ferme le tube en lecture, on remplace la sortie standard par le tube puis on ferme le tube en écriture.

— La commande est au moins la deuxième du pipeline : on ferme l'ancien tube en écriture, on remplace l'entrée standard par l'ancien tube puis on ferme l'ancien tube en lecture.

Cependant, il faut gérer les tubes dans le père. En effet, il ne faut pas que le père reste rédacteur pour le tube. Sinon, la lecture est bloquante. Il doit cependant rester lecteur pour que la commande suivante puisse remplacer son entrée standard par la sortie du tube.

Remarque : La conception ainsi réalisée ne change donc pas le mécanisme de lancement des commandes.

Résultat pour 2 terminaux ouverts :

```
(/home/ave9761/git_perso/1a/sec/minishell) > who | grep ave9761 | wc -l
Handler : Fils 220861 termine avec le code 0
Handler : Fils 220862 termine avec le code 0
2
Handler : Fils 220863 termine avec le code 0
(/home/ave9761/git_perso/1a/sec/minishell) >
```

VI – Remarques supplémentaires

On fait bien attention à ne pas faire de `printf()` pour ce qui ne concerne pas les commandes (informations, erreurs...). Sinon, on risque d'injecter des données dans les canaux `stdin` (0) et `stdout` (1).

Dans le fils, il faut ajouter toutes les commandes avant le `exec`. Sinon, elles ne sont jamais exécutées et remplacées par le code de la commande lancée.